

CPU Utilization as a Software-Only Thermal Proxy for Multi-Tenant Edge AI Inference: A 13-Hour Characterization and Coupling Law on Raspberry Pi 5

Manu Nicholas Jacob

Abstract—Thermal-aware control of edge AI inference usually presumes instrumented power sensing, which most deployed single-board computers (SBCs) lack. We ask a narrower, practical question: on a current-generation SBC running real inference, how well does *CPU utilization alone*, a fully software-observable signal, predict SoC temperature, and is the relationship stable enough to control against? From a 13-hour, four-block campaign on Raspberry Pi 5 (232,136 samples at 5 Hz: characterization, load sweeps, eight two-tenant configurations, and a six-hour stability run) we obtain a cross-validated *CPU–thermal coupling law*, $T \approx 45.6^\circ\text{C} + 0.175^\circ\text{C}/\% \cdot U$ ($R^2 = 0.93$), with leave-one-block-out error of $0.84\text{--}2.45^\circ\text{C}$ across four blocks of differing workload composition, a first-order die response of $\tau \approx 1\text{--}2$ s, and a slope that a six-month spot recalibration reproduces within 5%. Compared like-for-like against the standard structures (lumped-RC, autoregressive, and multi-signal models), the law matches every sensor-free, actuatable alternative (all collapse to $1.6\text{--}1.8^\circ\text{C}$), while sensor-initialized forecasters are $2.7\times$ tighter one step ahead, an advantage that decays within one time constant: rolled forward from the sensed temperature, an RC model is no more accurate than the law beyond a 5 s horizon. The law converts a temperature budget into a utilization budget with no power instrumentation. We release the runtime and full dataset.

Index Terms—Edge AI, thermal management, Raspberry Pi, multi-tenant inference, empirical characterization.

I. INTRODUCTION

EDGE AI deployments increasingly run continuous neural-network inference on small single-board computers (SBCs) in always-on perception pipelines, under a junction-temperature ceiling enforced by modest cooling, a fixed pool of cores shared among concurrent workloads, and application latency requirements. The thermal constraint integrates load over time: sustained inference heats the SoC toward a firmware throttle threshold where frequency is cut and latency degrades.

Thermal-aware runtime control needs a model that predicts temperature from observable signals. The natural predictor is board power, and a large body of work assumes an instrumented power rail; most deployed SBCs ship none. The

practical question is what temperature can be predicted from with only software-observable telemetry.

We answer it empirically for Raspberry Pi 5, the current-generation reference SBC: CPU utilization, read from the operating system, predicts SoC temperature through a simple linear law that transfers across workloads and time. Four contributions:

- **A cross-validated coupling law** for Raspberry Pi 5, $T \approx 45.6 + 0.175 U$, validated leave-one-block-out across four blocks of differing model, resolution, request rate, and tenancy (§IV).
- **Its dynamics**: a sensed response of $\tau \approx 1\text{--}2$ s under slower package and heatsink components, fast enough that the steady-state law is the right model at second-scale control ticks (§V).
- **A like-for-like comparison** against lumped-RC, AR/ARX, and multi-signal models under one protocol (§VI).
- **A multi-tenant characterization** over eight two-tenant pairs, and an open runtime plus 232,136-sample dataset (§VII, §IX).

Scope: no board power was measured (no sensor was attached; the law is the software-only alternative this absence motivates), and the governor was pinned to performance, so the law characterizes the fixed-frequency regime; both points return in §X.

II. RELATED WORK

DNN inference on edge SBCs. Efficient architectures [1], optimized runtimes [2], and edge benchmarks [5] characterize inference latency on SBCs, but none provide a reusable utilization-to-temperature model for control.

Thermal- and power-aware computing. Dynamic thermal management [3], [4] and DVFS actuate frequency from a privileged runtime; we target the complementary application layer, allocating work with the governor held fixed, the configuration a latency-oriented deployment pins anyway (§III).

Software thermal and power estimation. The established structures are: performance-counter power models feeding a lumped RC thermal model in the HotSpot methodology [7], as in Isci and Martonosi’s runtime power monitoring [6] and

M. N. Jacob received the dual B.S. degree in Electrical Engineering and Computer Engineering from the University of Massachusetts Amherst, Amherst, MA, USA, in 2025 (e-mail: manunicholasjacob@gmail.com; ORCID: 0009-0007-6589-6572).

Runtime, configurations, raw telemetry, and analysis scripts are released at <https://github.com/manunicholasjacob/pi5-thermal-proxy>.

Lee and Skadron’s counter-driven temperature sensing through HotSpot [8]; autoregressive forecasters fit to the temperature sensor itself, as in Coskun et al.’s ARMA-based proactive management [9]; and calibrated utilization-based power models on battery devices such as PowerTutor [10]. What is missing for deployed SBCs is a quantitative, cross-validated utilization-to-temperature law measured on current hardware under real inference, published with its dataset; §VI evaluates representatives of each structure above under one protocol.

III. MEASUREMENT METHODOLOGY

A. Platform

Raspberry Pi 5 (Broadcom BCM2712, quad-core Arm Cortex-A76 @ 2.4 GHz, the official active cooler), 64-bit Raspberry Pi OS, ONNX Runtime CPU execution provider. SoC temperature is read from `vcgencmd/sysfs`; CPU and memory utilization from the OS. The CPU governor is pinned to `performance`, so core frequency is a constant 2.4 GHz throughout (we verified a single frequency value across all samples); the campaign therefore characterizes the fixed-frequency operating regime typical of a latency-oriented inference deployment.

B. Runtime and telemetry

A Python runtime hosts ONNX Runtime workers and a telemetry thread sampling temperature, CPU utilization, and memory at 5 Hz to Apache Parquet; blocks run sequentially.

C. Experiment design

The campaign comprises four blocks over 13 continuous hours: **Block 1**, single-workload characterization (MobileNetV2 and ResNet-18 at two resolutions, fixed rate); **Block 2**, a MobileNetV2 load sweep populating the utilization–temperature space; **Block 3**, eight ordered pairs of two concurrent ONNX workloads (SqueezeNet, MobileNetV2, ResNet-18, EfficientNet-lite0, YOLOv5n) on four cores; **Block 5**, a six-hour continuous MobileNetV2 run testing for drift (labels follow the released logs; the campaign’s configured Block 4, a controller-policy comparison, was not executed).

D. Dataset provenance and de-duplication

The raw capture contains nine Parquet files. Redundant collector processes started up to ~90 s apart recorded each block (three for Block 1, two for each other block) at the same 4.95 Hz effective rate. We de-duplicate to one collector per run, yielding **232,136 unique time-aligned samples** (the naive sum over all nine files, 471,827, double-counts). All results below use the de-duplicated set. This provenance detail is stated so that the released dataset is interpreted correctly.

E. What we measure and what we do not

We report temperature, utilization, and memory; no power sensor was read, and rather than publish an unvalidated surrogate we omit power entirely (§X). Per-inference latency was not logged, so §VII is a thermal, utilization, and memory characterization.

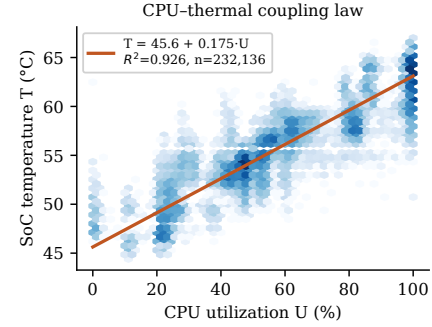


Fig. 1. CPU–thermal coupling on Raspberry Pi 5 (232,136 samples, log-density hexbins). SoC temperature is linear in CPU utilization with $R^2 = 0.93$; the fitted law is Eq. (1).

IV. THE CPU–THERMAL COUPLING LAW

A. Global fit

Across all 232,136 samples, SoC temperature is a strong linear function of CPU utilization (Fig. 1):

$$T = 45.63^\circ\text{C} + 0.1748 \frac{^\circ\text{C}}{\%} \cdot U, \quad (1)$$

with Pearson $r = 0.962$, $R^2 = 0.926$, and a residual standard deviation of 1.33°C . Consecutive samples are strongly autocorrelated, so we report uncertainty from a moving-block bootstrap (5-minute blocks): the slope’s 95% interval is $[0.168, 0.182]^\circ\text{C}/\%$. Restricting the fit to the interior 30–90% utilization band gives slope 0.151 with $R^2 = 0.70$; the pooled R^2 partly reflects the idle and saturation clusters the campaign design visits. Equation (1) says that each additional percent of aggregate CPU utilization raises steady-state SoC temperature by about 0.175°C , from an idle intercept near 45.6°C .

B. Cross-block validation

A law useful for control must generalize beyond the workload that produced it. We refit Eq. (1) in a leave-one-block-out protocol: train on three blocks, predict the held-out block. Prediction root-mean-square error (RMSE) ranges from 0.84°C (stability block) to 2.45°C (single-workload block): one global law predicts every held-out block within 2.5°C , across blocks that differ in model, resolution, request rate, and tenancy. The folds are unequal (7.6k–110k samples) and share one session and ambient trajectory, so this tests workload transfer; time and ambient transfer rest on the recalibration below and on re-measuring the intercept at deployment. Within-block slope estimates scatter more widely (0.13 – $0.21^\circ\text{C}/\%$, Fig. 2): each block occupies its own limited band of utilization (the load sweep is bimodal near idle and saturation; the multi-tenant block rarely leaves 42–100%), so a within-block fit has little leverage on the slope and folds slow ambient drift into it. We therefore take the pooled fit as the platform constant and validate it out of sample twice: by the leave-one-block-out transfer above, and by the independent six-month recalibration below, which reproduces the slope within 5%.

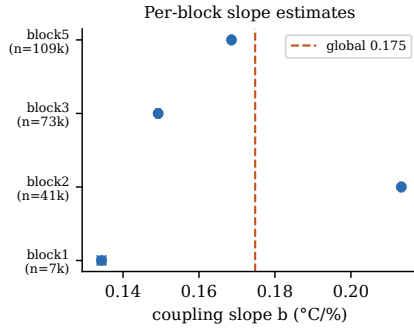


Fig. 2. Per-block slope estimates with 95% confidence intervals against the pooled fit (dashed). Within-block estimates scatter because each block spans a limited utilization band and folds ambient drift into the slope; the pooled slope is validated out of sample by held-out transfer ($\leq 2.45^\circ\text{C}$) and a six-month recalibration (within 5%).

C. Portability across time and ambient

A six-month spot check: recalibrating on the same unit in September 2026 (different season, ambient, and background state; a 150 s three-tenant soak against a 45 s idle read) reproduces the slope within 5% (0.183 vs. $0.175^\circ\text{C}/\%$) and the intercept within 0.1°C . This is a two-point check whose soak undersamples the minutes-scale component, so we read it as consistent with slope transfer rather than proof of it; the intercept must in any case be re-measured at deployment, a 45-second idle read.

D. Interpretation for control

Equation (1) is directly actionable. A controller that knows a thermal budget $T_{\max}$ can invert the law to a *utilization budget*, $U_{\max} = (T_{\max} - 45.63)/0.1748$, with U the system-wide utilization on the operating system’s 0–100 scale. Full load projects to a steady state of 63.1°C , so a cap at the 75°C firmware-throttle level cannot bind at this ambient with this cooling; the campaign’s observed maximum was 67.0°C (§VIII). Operator guard-band caps below 63°C do bind, and the inversion turns each into a concrete budget: $U_{\max} \approx 54\%$ for a 55°C cap. Computing it requires no power sensor: utilization is read from the OS. Because the budget guards an upper limit, the residual upper tail matters more than the RMSE: per-block p99 residuals are 2.0 – 3.7°C and brief multi-tenant transients reached $\sim 10^\circ\text{C}$ above the law, so an inverted budget should carry a guard margin sized to that tail, not to the mean error.

V. THERMAL DYNAMICS

Equation (1) is a steady-state relationship; a controller also needs the response speed. Two estimators put the sensed response on the order of a second: regressing the per-sample temperature increment on the gap to the law’s steady-state target over the six-hour stability run gives $\tau \approx 0.6$ s under a first-order model (biased low by sensor quantization at 5 Hz), and the lumped-RC grid search of §VI selects $\tau = 1$ – 2 s on held-out blocks; transients decay within a few seconds. Beneath this sit slower components: a ~ 10 s exponentially

TABLE I
LEAVE-ONE-BLOCK-OUT RMSE ($^\circ\text{C}$, MEAN OVER HELD-OUT BLOCKS).
“WHAT-IF” = CAN PREDICT THE EFFECT OF A PROPOSED ALLOCATION CHANGE.

Model	Signals	What-if	1-step	Sensor-free
Coupling law (1)	U	yes	1.76	1.76
+ EWMA (~ 10 s)	U	yes	1.64	1.64
Multi-signal static	U , mem	yes	1.65	1.65
Lumped RC ($\tau=1$ – 2 s)	U (+ T)	yes	0.67	1.71
ARX(1)	T , U	rollout	0.66	1.71
AR(1)	T	no	0.65	–

weighted moving average (EWMA) of utilization improves leave-one-block-out RMSE by 0.12°C (§VI), consistent with a package-scale pole, and soak behavior shows the operating point drifting over minutes (heatsink mass), which is why calibration requires a soak rather than an idle read. The consequence for control: at ticks of a second or slower, an allocation’s thermal effect is substantially realized within a few ticks ($3\tau \approx 3$ – 6 s), so the steady-state law is the right model; only sub-second control would need the transient term.

VI. COMPARISON WITH STANDARD PREDICTION STRUCTURES

The natural question for any proxy is how it compares with the established alternatives. We implement the standard structures from §II on the released dataset and evaluate all of them under the identical leave-one-block-out protocol: a HotSpot-class first-order lumped RC model driven by utilization [8] (time constant by grid search, steady-state gain by least squares); a Coskun-class autoregressive forecaster AR(1) on the temperature sensor [9] and its ARX extension with utilization as the exogenous input; a multi-signal static regression on utilization plus memory occupancy (the counter-model structure of [6] restricted to signals a stock SBC exports); and an exponentially-filtered variant of our own law. Table I reports both tasks that matter: 200 ms-ahead forecasting *with* the temperature sensor in the loop, and sensor-free prediction from software signals alone (the input a sensorless allocation controller actually consumes), obtained from the dynamic models by rolling them out on utilization without temperature feedback.

With the sensor available, short-horizon forecasters are $2.7\times$ tighter (0.65 vs. 1.76°C at one step), but they answer the wrong question for allocation: an AR-family model has no term through which a proposed duty-cycle change enters, and it requires the sensor whose absence motivates a proxy. Every sensor-free, actuatable structure instead lands in the same 1.6 – 1.8°C band. The operationally relevant middle case is a what-if initialized from the sensed temperature: rolling the RC model forward from the current reading under a proposed utilization gives 1.04°C at a 1 s horizon but 1.65°C by 5 s and parity with the law beyond (1.70 vs. 1.81°C at 10 s), the advantage decaying as $e^{-h/\tau}$. The board’s temperature sensor is present and useful for feedback; what the law removes is state estimation and per-platform dynamic fitting, at no accuracy cost for allocation horizons beyond a few seconds.

TABLE II
MULTI-TENANT SUMMARY (BLOCK 3, EIGHT TWO-TENANT PAIRS, 73,508
SAMPLES). COUPLING-LAW PREDICTION USES EQ. (1) AT THE OBSERVED
MEAN UTILIZATION.

Quantity	Value
Concurrent workload pairs	8
Mean CPU utilization	78.6%
Mean SoC temperature (observed)	59.3 °C
Mean SoC temperature (Eq. (1) prediction)	59.4 °C
Mean resident memory	1,162 MB
Thermal-budget violations (>75 °C)	0

Rolling out the dynamic RC or ARX models on utilization without temperature feedback recovers no accuracy over the static law, because with $\tau \approx 1\text{--}2\text{ s}$ a rollout converges to the law’s steady state within a few seconds, and transients occupy a negligible share of the 13-hour campaign; the richer static models buy 0.1 °C. On this class of platform the single-signal law is the accuracy frontier for sensor-free thermal prediction, at two constants, and the frontier is adequate under these workloads: a 1.6–1.8 °C band against the 8 °C minimum margin observed in §VIII.

VII. MULTI-TENANT CHARACTERIZATION

Block 3 runs eight ordered pairs of two concurrent ONNX workloads on the four cores, spanning SqueezeNet, MobileNetV2, ResNet-18, EfficientNet-lite0, and YOLOv5n (73,508 samples; Table II). A law fitted only on the other blocks predicts this block with 1.64 °C RMSE (the leave-one-block-out fold of §VI), evidence that the single law also governs the multi-tenant regime. No pair exceeded the thermal budget (maximum 65.9 °C); the resource that binds first under two tenants is memory (mean resident 1,162 MB of the board’s 2 GB) and core count, not temperature.

VIII. CONSTRAINT HEADROOM

Across the 13-hour campaign, SoC temperature ranged 44.4–67.0 °C with **zero** excursions above a 75 °C budget, and the six-hour stability block showed no monotonic drift. For the workloads tested with this cooling, thermal control on Raspberry Pi 5 is therefore *preventive* rather than reactive; Eq. (1) predicts the utilization at which a budget would bind.

IX. REPRODUCIBILITY ARTIFACT

We release the runtime, configurations, the de-duplicated dataset, and scripts that regenerate every number, figure, and Table I; the schema and de-duplication are documented.

X. LIMITATIONS

No power measurement. The campaign includes no board-power data. We note for precision that Raspberry Pi 5 does expose power-management-IC (PMIC) rail telemetry in software (`vcgencmd pmic_read_adc`); this campaign pre-dates our use of that interface, and validating the law against PMIC power is future work. The released dataset originally carried placeholder power fields written by the collector; they

contained no measurement and have been removed from the artifact, with a note documenting the removal.

Fixed frequency. The governor was pinned to performance, so Eq. (1) characterizes the constant-2.4-GHz regime. Under a dynamic governor, frequency becomes a second thermal driver and the single-variable law would need a frequency term.

Single board and ambient. Results are for one Raspberry Pi 5 unit under laboratory ambient with the official active cooler. The intercept (idle temperature) is ambient-dependent; the slope is expected to be more portable but is not verified across units or ambients here.

No per-inference latency. The telemetry does not include per-request latency, so multi-tenant interference is characterized thermally and in memory, not in latency or throughput.

XI. CONCLUSION

On Raspberry Pi 5, CPU utilization predicts SoC temperature through a tight, cross-validated linear law, $T \approx 45.6 + 0.175 U$ ($R^2 = 0.93$), predicting held-out experiment blocks within 2.5 °C, transferring across six months within 5% in slope, with a 1–2 s sensed response that keeps the steady-state law valid at control timescales. Compared like-for-like against the standard prediction structures, the law matches every actuatable alternative at allocation horizons; the sensor’s one-step advantage decays within a time constant. It lets an application-level controller invert a temperature budget to a utilization budget with no power sensor. We release the runtime and the full 232,136-sample dataset.

ACKNOWLEDGMENT

No external funding was received for this work. All experiments, data, analyses, and conclusions are the author’s own. Generative AI (Claude, Anthropic) assisted with manuscript editing, $\LaTeX$, and some analysis (the model comparison in §VI); all of its output was reviewed and verified by the author.

REFERENCES

- [1] M. Sandler *et al.*, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” in *Proc. IEEE/CVF CVPR*, 2018, pp. 4510–4520.
- [2] ONNX Runtime developers, “ONNX Runtime,” 2024. [Online]. Available: <https://onnxruntime.ai>
- [3] D. Brooks and M. Martonosi, “Dynamic Thermal Management for High-Performance Microprocessors,” in *Proc. HPCA*, 2001, pp. 171–182.
- [4] K. Skadron *et al.*, “Temperature-Aware Microarchitecture,” in *Proc. ISCA*, 2003, pp. 2–13.
- [5] C. Banbury *et al.*, “MLPerf Tiny Benchmark,” in *Proc. NeurIPS Datasets Benchmarks*, 2021.
- [6] C. Isci and M. Martonosi, “Runtime Power Monitoring in High-End Processors: Methodology and Empirical Data,” in *Proc. IEEE/ACM MICRO-36*, 2003, pp. 93–104.
- [7] W. Huang *et al.*, “HotSpot: A Compact Thermal Modeling Methodology for Early-Stage VLSI Design,” *IEEE Trans. VLSI Syst.*, vol. 14, no. 5, pp. 501–513, 2006.
- [8] K.-J. Lee and K. Skadron, “Using Performance Counters for Runtime Temperature Sensing in High-Performance Processors,” in *Proc. IEEE IPDPS Workshops (HP-PAC)*, 2005.
- [9] A. K. Coskun, T. S. Rosing, and K. C. Gross, “Proactive Temperature Management in MPSoCs,” in *Proc. ACM/IEEE ISLPED*, 2008, pp. 165–170.
- [10] L. Zhang *et al.*, “Accurate Online Power Estimation and Automatic Battery Behavior Based Power Model Generation for Smartphones,” in *Proc. IEEE/ACM/IFIP CODES+ISSS*, 2010, pp. 105–114.